CustomerSegment ML

CUSTOMER SEGMENTATION USING K-MEANS CLUSTERING

PROJECT REPORT

Submitted by

HERSHLY BOSE J [RA2411026040096]

DEENA DARSHAN M [RA2411026040097]

LIJO Y [RA2411026040074]

Under the guidance of

S.POORNIMA

(Assistant Professor (JrG), Department of CSE - Emerging Technologies)

in partial fulfillment for the Course

of

21CSC305P - MACHINE LEARNING

in

COMPUTER SCIENCE AND ENGINEERING

WITH A SPECIALIZATION IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(EMERGING TECHNOLOGIES)

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

VADAPALANI CAMPUS

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled "**CustomerSegment ML: Customer Segmentation Using K-Means Clustering**" is the Bonafide work of **"HERSHLY BOSE J [RA2411026040096]", "DEENA DARSHAN M [RA2411026040097]", "LIJO Y [RA2411026040074]"** who carried out a project work in 21CSC305P - Machine Learning under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE	**SIGNATURE**
Ms. S.Poornima, M.Tech, (Ph.D),	**Dr. P. Chitra, ME, Ph.D,**
Assistant Professor(Jr.G)	Professor & HOD
Dept. of CSE	Dept. of CSE
(Emerging Technologies)	(Emerging Technologies)

ACKNOWLEDGEMENTS

We express our humble gratitude to our Honorable Chancellor **Dr. T. R. Paarivendhar,** Pro Chancellor (Administration), **Dr. Ravi Pachamuthu,** Pro Chancellor (Academics) **Dr. P. Sathyanarayanan,** Pro Chancellor (Admin) for the facilities extended for the completion of the minor project work.

We would record our sincere gratitude to our Vice Chancellor, **Dr. C. Muthamizhchelvan** and Registrar, **Dr. S. Ponnusamy,** for their support in completing our minor project work by giving us the best of academic excellence support system in place. We extend our sincere thanks to our Director Academics, **Dr.Rama S Komaragiri** & Director Administration **Dr. Venkatesh Babu** of SRM Group. We extend our sincere thanks to our Dean Faculty of Engineering and Technology - **Dr. C.V. Jayakumar,** and Vice Principal - Academics **Dr. C. Gomathy** and Vice Principal - Examination - **Dr. S. Karthikeyan** for their invaluable support.

We wish to thank **Dr. P. Chitra**, Professor & Head, Department of Computer Science & Engineering - Emerging Technologies, SRM Institute of Science and Technology, Vadapalani Campus for her valuable suggestions and encouragement throughout the period of the minor project work and the course.

We extend our gratitude to our course faculty, **S.Poornima** Assistant Professor (Jr.G), Department of Computer Science & Engineering (Emerging Technology), SRM Institute of Science and Technology, Vadapalani Campus for providing us an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. We sincerely thank all the faculty and students of the department who have directly or indirectly helped our minor project. Finally, we would like to thank our parents, our family members and our friends for their unconditional love, constant support, and encouragement.

TEAM MEMBERS:

HERSHLY BOSE J [RA2411026040096]

DEENA DARSHAN M [RA2411026040097]

LIJO Y [RA2411026040074]

ABSTRACT

Every retailer serves customers who behave very differently. Some earn well and spend freely, some earn well but rarely spend, and others stretch a modest income to follow trends. Treating all of them with the same promotion wastes budget and annoys the people it is meant to attract. **Customer segmentation** solves this problem by dividing a customer base into groups whose members are similar to each other and different from the members of other groups. This project, **CustomerSegment ML**, delivers a complete, working machine-learning system that performs this segmentation automatically on any customer file a user uploads.

The system is built around **K-Means clustering**, an unsupervised learning algorithm that needs no labelled data. A customer CSV is uploaded through a responsive **Next.js** dashboard and sent to a **FastAPI** service. The service detects the income and spending columns (including the popular Kaggle *Mall Customers* headers), discards invalid rows, standardises both features with z-score scaling, sweeps K from 2 to 8, and recommends the number of clusters with the highest **silhouette score**. K-Means is then fitted with k-means++ seeding and ten restarts, centroids are converted back to real units, and each cluster is automatically given a business persona and a marketing strategy. A prediction endpoint places any new customer into the nearest segment in real time.

Evaluated on the 200-customer Mall Customers dataset, the system recommends **K = 5** and reaches a silhouette score of **0.555**, a Davies-Bouldin index of **0.572** and a Calinski-Harabasz index of **248.6**. In a head-to-head comparison against twelve alternative and baseline methods, the proposed K-Means configuration ranks **first** on both the silhouette and Calinski-Harabasz criteria, ahead of Gaussian mixtures, three hierarchical variants, BIRCH, DBSCAN, Mini-Batch K-Means and simple rule-based tiers, and far ahead of a random assignment (silhouette −0.069). Its k-means++ and ten-restart design produced the same partition on every one of 30 random seeds, whereas single-run random initialisation fell as low as 0.347. The five discovered segments (Premium VIP Shoppers, Careful Savers, Trendsetters, and two value-seeking groups) are clear, balanced and directly usable for targeted marketing.

Keywords: customer segmentation, K-Means clustering, unsupervised learning, silhouette analysis, elbow method, StandardScaler, FastAPI, Next.js, marketing analytics.

TABLE OF CONTENTS

S. No.	TITLE	PAGE NO.
1	**CHAPTER 1: INTRODUCTION**	**6**
	1.1 Background of the study	6
	1.2 Problem Statement	6
	1.3 Objectives	7
	1.4 Scope and Significance	7
2	**CHAPTER 2: LITERATURE REVIEW**	**8**
	2.1 Related Work	8
	2.2 Comparison of Existing Techniques	9
3	**CHAPTER 3: METHODOLOGY**	**11**
	3.1 Data Collection	11
	3.2 Data Preprocessing	11
	3.3 Feature Selection / Extraction	12
	3.4 Model Selection	14
	3.5 Algorithm Used	15
	3.6 Model Training & Validation	16
4	**CHAPTER 4: SYSTEM DESIGN**	**18**
	4.1 Architecture Diagram	18
	4.2 Flow Chart	19
	4.3 Module Description	19
5	**CHAPTER 5: IMPLEMENTATION**	**21**
	5.1 Tools and Technologies Used	21
	5.2 Model Development Steps	21
6	**CHAPTER 6: RESULTS & DISCUSSION**	**23**
	6.1 Experimental Results	23
	6.2 Performance Comparison	29
	6.3 Result Analysis	31
7	**CHAPTER 7: CONCLUSION & FUTURE WORK**	**33**
	7.1 Summary of Findings	33
	7.2 Limitations	33
	7.3 Future Enhancements	33
8	**REFERENCES**	**34**
9	**APPENDICES**	**36**
	9.1 Dataset Description	36
	9.2 Code Snippets	36
	9.3 Additional Graphs & Outputs	38

CHAPTER 1: INTRODUCTION

1.1 Background of the study

Modern businesses collect more information about their customers than ever before. Loyalty cards, online accounts and point-of-sale systems record who buys, how often and how much. Yet raw data alone does not tell a marketing team what to do. A shopping mall with thousands of members cannot design a personal campaign for each one, and a single campaign for everybody ignores the obvious fact that customers are not alike. The classic answer, first formalised in marketing theory by Smith in 1956 [14], is **market segmentation**: split the market into a small number of homogeneous groups and design one strategy per group.

Traditionally, segments were drawn by hand using simple rules such as age brackets or income bands. These rules are easy to explain but they look at one variable at a time and rarely match how people actually behave. Two customers with the same income can have completely opposite spending habits. **Machine learning** offers a better route. Clustering algorithms look at several variables together and discover the natural groupings hidden in the data, without anyone having to define the groups in advance [13], [15].

Among clustering methods, **K-Means** [1], [2] is the most widely used in industry. It is simple to understand, fast enough to run on millions of records, and produces a *centroid* for each cluster that can be read as the profile of a typical member. Those centroids translate directly into business language, for example “high income, low spending”, which is exactly what a marketing manager needs.

This project, **CustomerSegment ML**, turns that idea into a usable product. Instead of a one-off notebook, it is a full-stack web application: a user uploads a customer file, watches the processing stages, and receives an interactive dashboard with the elbow curve, the clustered scatter plot, a profile and a marketing strategy for every segment, a live prediction form for new customers and a downloadable file of results. It was developed as the course project for 21CSC305P Machine Learning.

1.2 Problem Statement

Given a table of customers in which each row contains at least an **annual income** and a **spending score** (with an optional customer identifier and age), partition the customers into K groups such that customers inside a group are as similar as possible and customers in different groups are as different as possible, choose a suitable K automatically, describe each group in business terms, and assign any future customer to the correct group instantly. Formally this is an **unsupervised clustering** problem: there are no ground-truth labels, so the quality of a solution must be judged by internal measures of cohesion and separation.

Several practical difficulties make the problem more than a textbook exercise:

- **No labels.** Nobody knows the “correct” segments, so the system must decide the number of clusters and justify its choice using internal validity indices.

- **Different feature scales.** Income is measured in thousands of dollars while the spending score runs from 1 to 100. Without scaling, the feature with the larger numeric range would dominate the distance calculation.
- **Messy uploads.** Real files use different column names, contain blank or non-numeric cells and may be far larger than expected. The system has to validate and clean them without crashing.
- **Sensitivity to initialisation.** K-Means can converge to a poor local minimum if its starting centroids are badly placed, so a naive implementation may give different answers on different runs.
- **Interpretability.** A cluster number on its own means nothing to a business user; each segment must be explained and paired with an action.

1.3 Objectives

- Build an end-to-end pipeline that turns an uploaded customer CSV into labelled customer segments with no manual steps.
- Detect the required columns automatically, accept both the project's own headers and the Kaggle *Mall Customers* headers, and skip invalid rows with a clear report.
- Compute the elbow curve (WCSS) for K = 2 to 8 and recommend the K with the highest silhouette score.
- Train K-Means with k-means++ seeding and multiple restarts to obtain a stable, reproducible partition.
- Provide a real-time prediction service that assigns a new customer to the nearest centroid.
- Evaluate the model with silhouette, Davies-Bouldin and Calinski-Harabasz indices and compare it against alternative and baseline clustering methods.
- Present everything in a responsive, interactive web dashboard backed by a validated REST API and automated tests.

1.4 Scope and Significance

The scope of the project is a decision-support tool for retail and marketing analytics that works on two behavioural features, annual income and spending score, for files of 10 to 5,000 customers. It is intended for analysts, small businesses and students who need trustworthy segments without writing code. Within that scope the project contributes:

These are a reusable, tested segmentation engine (`model.py`), automatic model selection so the user never has to guess the number of clusters, human-readable personas and strategies, a clean REST API with schema validation and Swagger documentation, and a modern dashboard with charts, tables, a K slider, a prediction form and CSV export.

The significance of the work lies in closing the gap between a machine-learning algorithm and a business decision: segmentation that a manager can run on their own file in seconds, inspect visually and download is far more likely to be used than a notebook, and the same pipeline is a clear teaching example of unsupervised learning.

CHAPTER 2: LITERATURE REVIEW

Clustering is one of the oldest problems in statistics and data mining, and customer segmentation is one of its most common commercial uses. This chapter reviews the clustering algorithms that are relevant to the project, the methods used to choose the number of clusters and validate a partition, and the way segmentation has been applied in marketing. It ends with a structured comparison that positions CustomerSegment ML against the existing techniques.

2.1 Related Work

Partitional clustering: K-Means and its variants

K-Means was described by MacQueen in 1967 [1], while the iterative assign-and-update procedure that most libraries implement today is known as **Lloyd's algorithm** [2]. It minimises the within-cluster sum of squared distances and converges quickly, usually in a handful of iterations. Its main weakness is sensitivity to the starting centroids. Arthur and Vassilvitskii addressed this with **k-means++** [3], which spreads the initial centroids out by choosing each new seed with probability proportional to its squared distance from the seeds already chosen; they proved that this gives an expected solution within O(log K) of the optimum. **Mini-Batch K-Means** [12] trades a little accuracy for speed by updating centroids from small random batches, which is useful for web-scale data but unnecessary for a few thousand customers.

Hierarchical clustering

Agglomerative methods start with every point in its own cluster and repeatedly merge the two closest clusters. The definition of “closest” matters a great deal. **Ward's method** [8] merges the pair that causes the smallest increase in total within-cluster variance and therefore behaves much like K-Means. Complete and average linkage use the farthest and the average pairwise distances, while single linkage uses the nearest pair and is known to produce long, chained clusters. Hierarchical methods give a full dendrogram, which is informative, but they need $O(n^2)$ memory and cannot assign a new point without re-running the merge.

Density-based and model-based clustering

DBSCAN [9] groups points that lie in dense regions and labels isolated points as noise. It finds clusters of arbitrary shape and does not need K, but it needs a radius (eps) and a minimum neighbour count, and a single radius rarely fits clusters of different density. **Gaussian Mixture Models** fitted with the EM algorithm [10] assume each cluster is a Gaussian distribution and produce soft membership probabilities; they are more flexible than K-Means but have more parameters and can be unstable on small data. **BIRCH** [11] builds a compact clustering-feature tree and is designed for very large databases that do not fit in memory.

Choosing K and validating clusters

Because clustering has no labels, internal indices are used to judge quality. The **elbow method**, dating back to Thorndike [7], plots the within-cluster sum of squares against K and looks for the point where adding clusters stops paying off. The **silhouette coefficient** of Rousseeuw [4] compares, for

every point, its average distance to its own cluster with its average distance to the nearest other cluster, giving a score between −1 and +1. The **Davies-Bouldin index** [5] averages the worst-case ratio of within-cluster scatter to between-centroid distance (lower is better), and the **Calinski-Harabasz index** [6] is the ratio of between-cluster to within-cluster dispersion (higher is better). The **gap statistic** [23] compares WCSS with that of uniform random data. The **adjusted Rand index** [24] measures agreement between two partitions and is useful for checking stability across runs.

Segmentation in marketing

Marketing research has long used clustering to build segments from demographic, behavioural and transactional variables [15]. **RFM analysis** (recency, frequency, monetary value) [16] is a popular rule-based alternative that scores customers on three transaction measures. The Kaggle *Mall Customers* dataset [21], with income and spending score for 200 members, has become a standard benchmark for teaching and comparing segmentation methods; most published notebooks on it report five well-separated groups when K-Means is applied to the two behavioural features. Most of that work, however, stops at a static plot inside a notebook. Very few studies package the method as a validated service that accepts arbitrary uploads, chooses K automatically and explains its output.

2.2 Comparison of Existing Techniques

Table I compares representative techniques with the proposed CustomerSegment ML system in terms of their mechanism, advantages and limitations for the task of customer segmentation.

TABLE I
COMPARISON OF EXISTING TECHNIQUES WITH THE PROPOSED WORK

S. No.	Technique	Methodology	Advantages	Limitations Compared with the Proposed Work
1	Rule-based tiers / RFM [16]	Fixed thresholds on one or a few variables	Very simple to explain	Ignores joint structure; thresholds are arbitrary
2	K-Means, random init [1], [2]	Lloyd iterations from random seeds	Fast, centroid profiles	Unstable; can fall into poor local minima
3	k-means++ [3]	D^2 weighted seeding before Lloyd	Better, provable seeding	Still needs K chosen by the user
4	Mini-Batch K-Means [12]	Centroid updates from random batches	Scales to huge data	Slightly worse partitions on small data
5	Agglomerative (Ward / linkage) [8]	Bottom-up merging of clusters	Dendrogram, no seeding	$O(n^2)$ cost; cannot predict new points directly
6	DBSCAN [9]	Density reachability with eps radius	Arbitrary shapes, finds noise	Sensitive to eps; struggles with varied density
7	Gaussian Mixture (EM) [10]	Probabilistic mixture of Gaussians	Soft memberships, flexible shapes	More parameters; less stable on small data
8	BIRCH [11]	CF-tree summarisation then clustering	Memory efficient for big data	Threshold tuning; weaker on small sets
9	**Proposed CustomerSegment ML**	**Scaled K-Means, k-means++, 10 restarts, silhouette-selected K, personas, API + dashboard**	**Automatic K, stable, explainable, real-time prediction, usable by non-experts**	**Limited to income and spending features in this version**

Summary of the comparison

The literature shows that no single algorithm is best in every situation, but for compact, roughly spherical behavioural groups K-Means with good seeding is accurate, fast and uniquely easy to explain through its centroids. Its known weaknesses, the need to pick K and sensitivity to initialisation, can be handled by silhouette-based selection and by k-means++ with several restarts. CustomerSegment ML combines exactly these remedies with automatic data validation, persona generation and a real-time service, and it is evaluated against the alternative methods in Chapter 6 rather than assumed to be better.

CHAPTER 3: METHODOLOGY

3.1 Data Collection

CustomerSegment ML is deliberately **dataset-agnostic**: it ships with no built-in data and works on whatever customer file the user uploads. For development and evaluation the team used the public **Mall Customers** dataset published on Kaggle [21], which is the most widely used benchmark for this task. It describes 200 members of a shopping mall with five attributes, summarised in Table II.

TABLE II
SUMMARY OF THE EVALUATION DATASET

S. No.	Attribute	Type	Range / Values	Used for clustering
1	CustomerID	Identifier	1 to 200	No (kept for output)
2	Genre (gender)	Categorical	Female 112, Male 88	No
3	Age	Integer	18 to 70 years (mean 38.9)	No (shown in profiles)
4	Annual Income (k$)	Integer	15 to 137 (mean 60.6)	Yes
5	Spending Score (1-100)	Integer	1 to 99 (mean 50.2)	Yes

The **spending score** is assigned by the mall from purchasing behaviour and loyalty, so it summarises how actively a customer spends. The dataset contains no missing values and no duplicate rows. Because the problem is unsupervised, there is no train/test split of labels in the usual sense. Instead the whole file is clustered, and quality is measured with internal indices; stability is checked by re-running the algorithm with many random seeds, and the prediction service is checked with hand-made test customers whose correct segment is obvious.

To check that the system is not tied to this single file, the automated test suite also generates synthetic uploads with Kaggle-style headers, four known groups and a deliberately broken row. The system must detect the columns, skip the broken row, recommend K = 4 and recover the four groups.

3.2 Data Preprocessing

Uploaded files are often inconsistent, so preprocessing is designed to be tolerant of variation while being strict about what the model needs. The steps are:

1. **Size and encoding check.** The raw request body must be non-empty and at most 5 MB. It is decoded as UTF-8 with an optional byte-order mark, so files exported from Excel are accepted.
2. **Column detection.** Each header is lower-cased and stripped of every non-alphanumeric character, so `Annual Income (k$)`, `annual_income` and `AnnualIncome` all become `annualincomek` or `annualincome` and are matched to the income feature. The same rule matches spending score, age and customer ID.

3. **Type coercion.** Income, spending and age are converted to numbers; any value that cannot be converted becomes missing instead of raising an error.
4. **Cleaning.** Infinite values are replaced by missing values and every row with missing income or spending is dropped. The number of skipped rows is returned to the user.
5. **Size limits.** At least 10 valid customers are required so that K up to 8 is meaningful, and at most 5,000 are accepted to keep the dashboard responsive.
6. **Feature scaling.** Income and spending score are standardised with z-score normalisation.

Feature scaling deserves special attention because K-Means is entirely based on Euclidean distance. Each feature x is transformed with its mean μ and standard deviation σ:

$$z = (x - \mu) / \sigma \tag{1}$$

After this transformation both features have mean 0 and standard deviation 1, so a one-unit change means “one standard deviation” for income and for spending alike. Fig. 1 shows the data before and after scaling: the shape of the cloud is preserved, only the axes change.

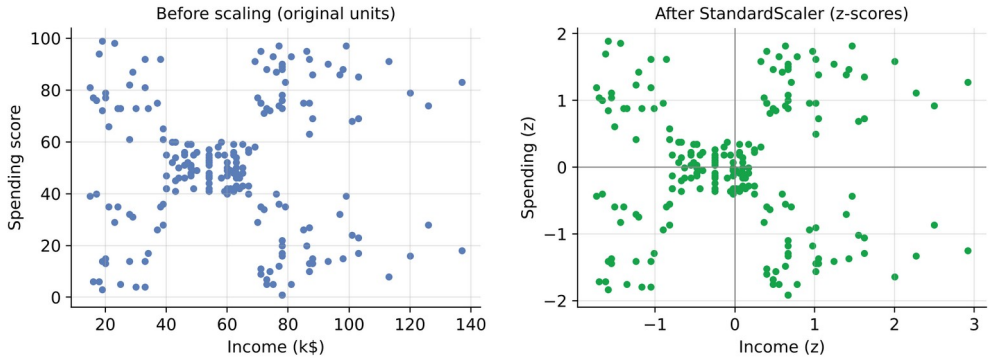


Fig. 1. Customer data before and after z-score standardisation

On the Mall dataset the two features happen to have similar spreads (σ ≈ 26.3 and 25.8), so scaling changes the result very little. On real uploads, however, income may be recorded in rupees or dollars rather than thousands, which would make it thousands of times larger than the score. Scaling makes the system robust to such files without any user action.

3.3 Feature Selection / Extraction

Exploratory analysis guided the choice of features. Fig. 2 shows the distribution of age, income and spending score. Age is skewed towards younger adults, income is moderately right-skewed with a few high earners, and spending score is spread across the whole range.

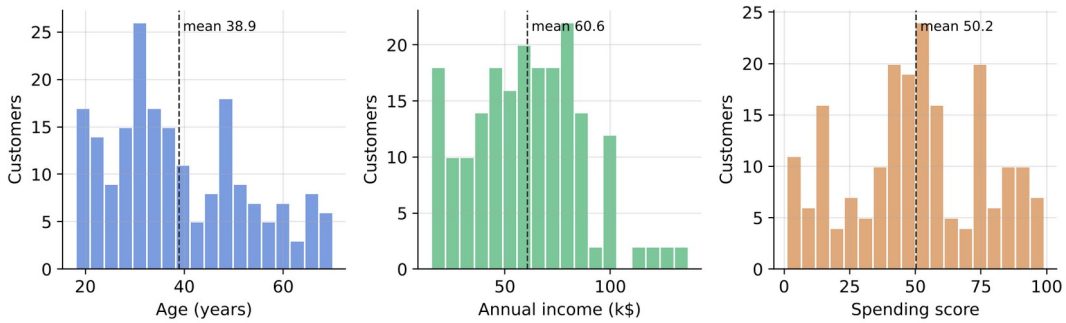


Fig. 2. Distributions of age, annual income and spending score

Fig. 3 shows the correlation matrix. Income and spending score are almost uncorrelated (r = 0.01), which means they carry independent information and together describe two different aspects of a customer: financial capacity and willingness to spend. Age is moderately negatively correlated with spending (r = −0.33), so it adds little that spending does not already reflect.

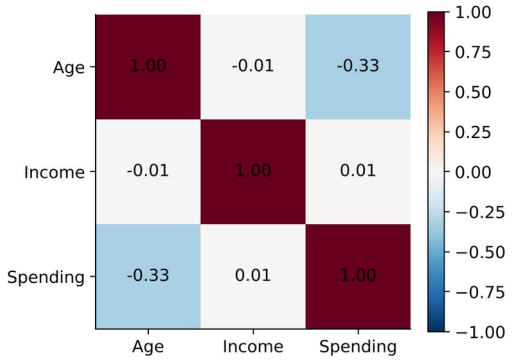


Fig. 3. Pearson correlation between age, income and spending score

The decisive evidence comes from the raw scatter plot in Fig. 4. In the income and spending plane the customers already form five visible groups: a dense centre and four corners. No such structure appears when age is included, because age is spread evenly across the groups. The project therefore uses **Annual Income** and **Spending Score** as the clustering features, keeps **Age** as a descriptive attribute in the cluster profiles, and does not use gender, which is a sensitive attribute that should not drive marketing decisions on its own.

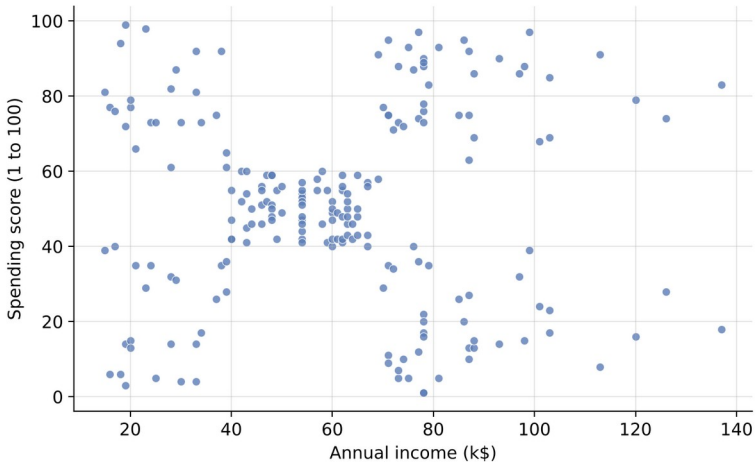


Fig. 4. Raw customers in the income and spending plane before clustering

Two features also have a practical advantage: every result can be drawn exactly on a two-dimensional chart, so business users can see why a customer belongs to a segment instead of trusting a black box.

3.4 Model Selection

Model selection has two parts: choosing the clustering algorithm and choosing the number of clusters K.

Choice of algorithm

K-Means was selected as the primary model for four reasons. First, the groups seen in Fig. 4 are compact and roughly round, which is exactly the shape K-Means is designed for. Second, its centroids are directly interpretable as the average income and spending of a segment. Third, a fitted K-Means model can classify a new customer in microseconds by finding the nearest centroid, which hierarchical methods and DBSCAN cannot do without refitting. Fourth, it is fast and deterministic once the random seed is fixed. The alternatives in Table I were not discarded on assumption; all of them are evaluated on the same data in Section 6.2.

Choice of K

K is chosen automatically by a sweep over K = 2 to 8, summarised in Fig. 5. For every K the system fits K-Means, records the WCSS for the elbow chart and computes the silhouette score. The **recommended K is the one with the highest silhouette score**, because silhouette rewards both tight clusters and wide gaps between them, while WCSS always decreases as K grows and therefore cannot choose K by itself.



Fig. 5. Automatic selection of the number of clusters

$$K^* = \arg\max S(K), \quad K \in \{2, 3, \ldots, 8\} \tag{2}$$

Davies-Bouldin and Calinski-Harabasz indices are computed as supporting evidence. The user can still override the recommendation with a slider in the dashboard, which is useful when the business prefers fewer or more segments.

3.5 Algorithm Used

The core algorithm is K-Means with k-means++ initialisation, as implemented by scikit-learn [17]. Given n scaled customers $x_1 \ldots x_n$ and a number of clusters K, K-Means looks for centroids $\mu_1 \ldots \mu_K$ and an assignment of every customer to one centroid. The distance between a customer and a centroid is the Euclidean distance:

$$d(x, \mu) = \sqrt{((x_1 - \mu_1)^2 + (x_2 - \mu_2)^2)} \tag{3}$$

Assignment step. Every customer is assigned to its nearest centroid:

$$c_i = \arg\min_j \| x_i - \mu_j \|^2 \tag{4}$$

Update step. Every centroid moves to the mean of the customers assigned to it:

$$\mu_j = (1 / |C_j|) \cdot \Sigma\, x_i , \quad x_i \in C_j \tag{5}$$

The two steps are repeated until the assignments stop changing. Each step can only lower the objective, the **within-cluster sum of squares (WCSS)**, also called inertia, so the procedure always converges:

$$J = \Sigma_{j=1}^{K} \Sigma_{i \in C_j} \| x_i - \mu_j \|^2 \tag{6}$$

k-means++ seeding. The first centroid is a random customer. Each further centroid is chosen at random with probability proportional to the squared distance $D(x)^2$ from the nearest centroid already chosen, so the seeds tend to be spread across the data:

$$P(x) = D(x)^2 / \Sigma_{x'} D(x')^2 \tag{7}$$

The whole procedure is run **ten times** (`n_init=10`) from different seeds and the run with the lowest WCSS is kept, with `random_state=42` making the result reproducible. Fig. 6 summarises the loop.

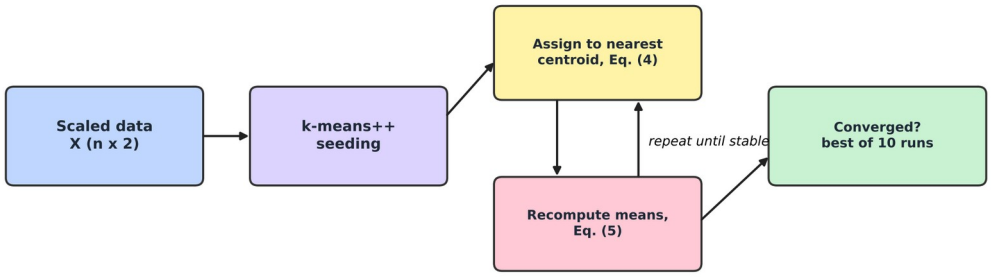


Fig. 6. K-Means with k-means++ seeding and multiple restarts

After fitting, centroids are mapped back into real units with the inverse of the scaling transform, so they can be shown as “$86.5k income, score 82.1”:

$$\hat{x} = \sigma \cdot z + \mu \tag{8}$$

The complete CustomerSegment ML algorithm is:

1. Read and validate the uploaded CSV; detect columns and drop invalid rows.
2. Extract the feature matrix X = [income, spending] and standardise it.
3. For K = 2 … 8: fit K-Means, store WCSS and silhouette.
4. Set the recommended K to the K with the highest silhouette.
5. Fit the final K-Means (k-means++, 10 restarts, seed 42) on the scaled data.
6. Inverse-transform the centroids and compute per-cluster count, share, mean age, mean income and mean spending.
7. Compare each cluster's mean income and spending with the overall means to assign a persona and a strategy.
8. Publish the fitted scaler, model and metadata together, so predictions never see a half-trained model.
9. For a new customer: scale the input with the fitted scaler, find the nearest centroid and return its persona and strategy.

3.6 Model Training & Validation

Because there are no labels, validation uses internal indices that measure how compact and how well separated the clusters are. For a customer i, let a(i) be its mean distance to the other members of its own cluster and b(i) its mean distance to the members of the nearest other cluster. The silhouette coefficient [4] is:

$$s(i) = (b(i) - a(i)) / \max\{ a(i), b(i) \} \tag{9}$$

A value near +1 means the customer sits deep inside its own cluster, a value near 0 means it lies on a boundary, and a negative value means it would fit better elsewhere. The overall score is the mean over all n customers:

$$S = (1 / n) \sum_{i=1}^{n} s(i) \tag{10}$$

The **Davies-Bouldin index** [5] compares, for each cluster, its average scatter σ_i with its distance to the most similar other cluster; lower values are better:

$$DB = (1 / K) \sum_{i=1}^{K} \max_{j \neq i} (\sigma_i + \sigma_j) / d(\mu_i, \mu_j) \tag{11}$$

The **Calinski-Harabasz index** [6] is the ratio of between-cluster dispersion B_k to within-cluster dispersion W_k, corrected for the number of clusters; higher values are better:

$$CH = [\operatorname{tr}(B_k) / (K - 1)] / [\operatorname{tr}(W_k) / (n - K)] \tag{12}$$

Training and validation follow the configuration in Table III. For uploads larger than 2,000 customers the silhouette is estimated on a random sample of 2,000 points, because its cost grows with the square of n. Stability is validated by repeating the fit with 30 different random seeds and measuring the spread of the silhouette score and the adjusted Rand index between runs [24].

TABLE III
TRAINING AND VALIDATION CONFIGURATION

Parameter	Value	Reason
Algorithm	sklearn.cluster.KMeans	Lloyd iterations, well tested
Initialisation	k-means++	Spread-out seeds, fewer bad optima
Restarts (n_init)	10	Keep the best of ten runs
Random state	42	Reproducible results
Features	Annual income, spending score	Independent behavioural signals
Scaling	StandardScaler (z-score)	Equal weight in Euclidean distance
K search range	2 to 8	Practical number of marketing segments
K selection rule	Maximum silhouette	Balances cohesion and separation
Silhouette sample	2,000 points if n > 2,000	Keeps $O(n^2)$ cost bounded
Input limits	10 to 5,000 rows, 5 MB	Meaningful clusters, responsive UI

CHAPTER 4: SYSTEM DESIGN

CustomerSegment ML follows a classic three-tier design: a presentation tier (the Next.js dashboard), an application tier (the FastAPI service) and a model tier (the segmentation engine with scikit-learn). Each tier has one responsibility and talks to the next through a small, typed interface, which makes every part easy to test and to replace.

4.1 Architecture Diagram

Fig. 7 shows the architecture. The user interacts with the dashboard in a browser. The dashboard calls the backend through a typed API client that handles errors and offline states. FastAPI validates every request and response with Pydantic schemas and forwards the work to a single `CustomerSegmentationModel` object, which wraps the scaler, the K-Means model and the persona rules. The uploaded data and the fitted model live in memory, so no customer data is written to disk.

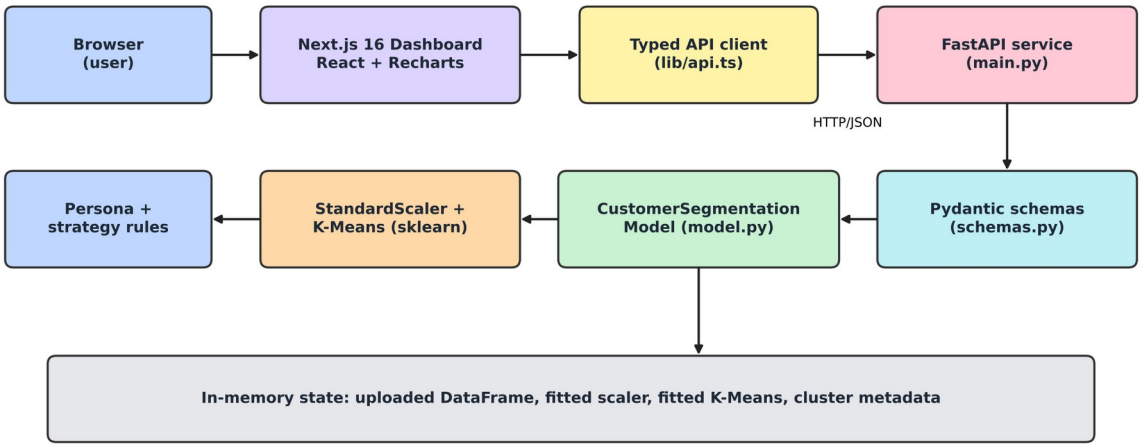


Fig. 7. Architecture of the CustomerSegment ML system

Fig. 8 shows the order of the API calls during a typical session. The upload is sent first; once the dataset is active the dashboard requests the elbow curve, trains the model with the recommended K and, whenever the user asks, predicts the segment of a new customer. Every endpoint except health and upload returns HTTP 409 until a dataset has been uploaded, so the client can never run the model on missing data.

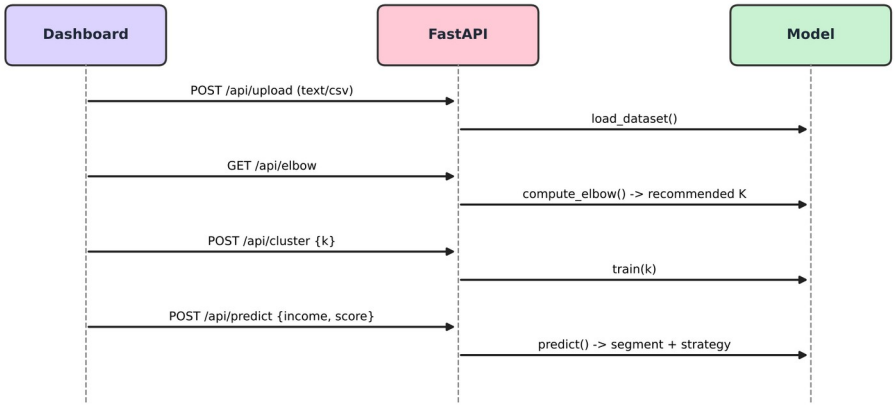


Fig. 8. Sequence of API calls between the dashboard, the API and the model

4.2 Flow Chart

The flow chart in Fig. 9 represents the complete process from an uploaded file to segmented, downloadable results.

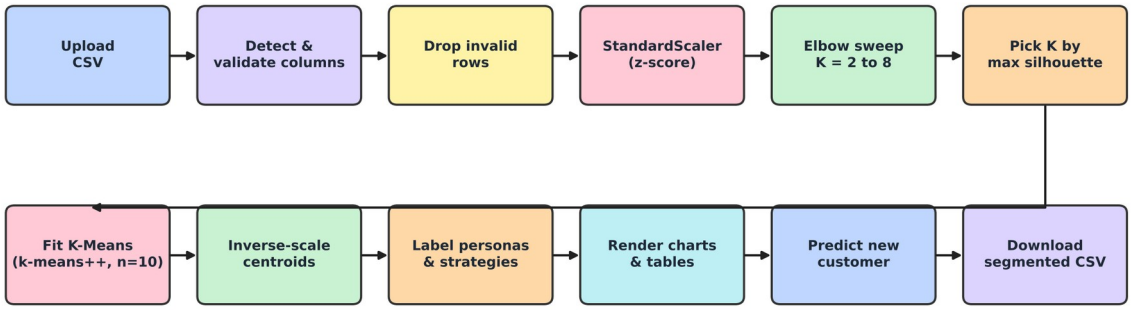


Fig. 9. Flow chart of the CustomerSegment ML pipeline

The process starts when the user uploads a CSV. The backend checks its size, detects the columns and removes invalid rows. The two features are standardised, the elbow sweep runs for K = 2 to 8 and the K with the best silhouette is selected. K-Means is fitted, its centroids are converted back to real units and every cluster receives a persona and a strategy. The dashboard then renders the elbow chart, the scatter plot, the cluster cards and the dataset table. From there the user can retrain with a different K, predict a new customer or download the segmented file.

4.3 Module Description

The system is divided into functional modules, each owning one stage of the workflow, as listed in Table IV.

TABLE IV
FUNCTIONAL MODULES OF CUSTOMERSEGMENT ML

Module	Description
Data module (`model.py`, load_dataset)	Column detection, numeric coercion, cleaning, size limits and replacement of the active dataset.
Model-selection module (`compute_elbow`)	Fits K-Means for K = 2 to 8, returns WCSS values and the silhouette-optimal K.
Training module (`train`)	Scales features, fits K-Means, inverse-transforms centroids, builds cluster statistics and the silhouette score.
Interpretation module	Compares each cluster with the overall means to produce one of four personas plus a description and a marketing strategy.
Inference module (`predict`)	Scales a new customer and returns the nearest cluster, persona, strategy and centroid.
API module (`main.py`)	FastAPI routes, CORS, size limits, dependency that blocks model calls before upload, error mapping to HTTP codes.
Schema module (`schemas.py`)	Pydantic models that validate every request (for example $2 \le K \le 8$) and response.
Dashboard module (`app/`)	Next.js components: upload with progress stages, dataset table, elbow chart, K slider, scatter plot, cluster cards, prediction form.
Test module (`test_api.py`)	End-to-end test of every endpoint, including error paths, on a synthetic upload.

The interpretation module uses a simple, transparent rule. A cluster whose mean income and mean spending are both at or above the overall averages is labelled **Premium VIP Shoppers**; high income with low spending gives **Careful Savers**; lower income with high spending gives **Trendsetters / Enthusiasts**; and low income with low spending gives **Budget / Value Seekers**. When K is larger than four, a group number is added so that every label is unique on screen. Because the rule is explicit, a marketing manager can always see why a cluster received its name.

CHAPTER 5: IMPLEMENTATION

5.1 Tools and Technologies Used

The implementation combines a Python machine-learning backend with a TypeScript web frontend. Table V lists the tools and their purpose.

TABLE V
TOOLS AND TECHNOLOGIES

Tool / Technology	Purpose
Python 3.12	Language of the machine-learning backend
scikit-learn [17]	KMeans, StandardScaler, silhouette and other cluster metrics
NumPy [18] / pandas [19]	Array operations, CSV parsing, cleaning and group statistics
FastAPI [22] + Uvicorn	Asynchronous REST API with automatic Swagger documentation
Pydantic v2	Request and response validation
Next.js 16 (App Router) + React	Interactive single-page dashboard
TypeScript	Type-safe API client and component props
Tailwind CSS v4	Responsive styling with light and dark themes
Recharts	Elbow line chart and cluster scatter plot in the browser
pytest + FastAPI TestClient	Automated end-to-end API tests
Matplotlib [20]	Figures and charts in this report
Git and GitHub	Version control and collaboration

The backend is about 600 lines of Python across three modules and a test file, and the frontend is about 2,700 lines of TypeScript across eight components, a typed API client and the main page.

5.2 Model Development Steps

Development followed the same order as the data flow, so that each stage could be tested before the next was built.

Step 1: Explore the data

The Mall Customers file was loaded with pandas and inspected with summary statistics, histograms, a correlation matrix and scatter plots (Figs. 2 to 4). This confirmed that income and spending score form clear groups and justified the feature choice.

Step 2: Build the data loader

`load_dataset` was written to normalise column names, coerce types, drop invalid rows and enforce the 10 to 5,000 row limits. It returns a summary of total, processed and skipped rows and the detected columns, which the dashboard shows after every upload.

Step 3: Implement scaling and the elbow sweep

`compute_elbow` fits a fresh `StandardScaler` and runs K-Means for every K from 2 to 8, collecting WCSS and silhouette. The recommended K is the argmax of the silhouette list.

Step 4: Train and interpret

`train` fits the scaler and K-Means into local variables, computes centroids in real units, cluster sizes, shares and means, generates personas and strategies, and only then publishes the new scaler, model and metadata together. This prevents a prediction request that arrives during training from seeing a half-built model.

Step 5: Add real-time prediction

`predict` scales the income and spending of a new customer with the fitted scaler and calls `model.predict`. If the user predicts before training, the model trains itself with K = 4 first.

Step 6: Expose the REST API

Six endpoints were implemented in FastAPI (Table VI). Uploads are read from the raw request body, so no multipart library is needed, and heavy work runs in a thread pool to keep the server responsive.

TABLE VI
REST API ENDPOINTS

Method	Endpoint	Description
GET	/api/health	Service status and number of loaded customers
POST	/api/upload	Upload a CSV (text/csv, max 5 MB) and make it the active dataset
GET	/api/dataset	Uploaded customer records, optional ?limit=
GET	/api/elbow	WCSS for K = 2 to 8 and the silhouette-optimal recommended K
POST	/api/cluster	Train K-Means with {k} (2 to 8); returns metrics, clusters, points, centroids
POST	/api/predict	Classify {income, spending_score, age?} into a segment

Step 7: Build the dashboard

The Next.js dashboard was built as small client components: a navigation bar with a live API status indicator, an upload panel with processing stages and CSV export, a searchable dataset table, the elbow chart, a K slider, the scatter plot with centroids, cluster cards and the prediction form. All API calls go through one typed client that converts network failures into friendly messages.

Step 8: Test and evaluate

An automated test uploads a synthetic file with four groups and a broken row and checks every endpoint, including the 409 responses before upload and the validation error for K = 10. Finally, the evaluation in Chapter 6 was run on the Mall dataset using the project's own `CustomerSegmentationModel` class, so the reported numbers are exactly what users see in the dashboard.

CHAPTER 6: RESULTS & DISCUSSION

6.1 Experimental Results

All results in this chapter were produced by running the project's own code on the 200-customer Mall Customers dataset. Table VII summarises the configuration and the headline values.

TABLE VII
SYSTEM CONFIGURATION AND RECORDED VALUES

S.No.	Evaluation Parameter	Recorded Value	Significance
1	Dataset	Mall Customers, 200 rows	Standard segmentation benchmark
2	Rows processed / skipped	200 / 0	Columns detected automatically
3	Features	Annual income, spending score	Two independent behaviours
4	Recommended K	5	Highest silhouette in K = 2 to 8
5	Silhouette score	0.555	Good cohesion and separation
6	Davies-Bouldin index	0.572	Lowest of all K values
7	Calinski-Harabasz index	248.6	Strong between-cluster spread
8	WCSS (scaled space)	65.57	Clear elbow at K = 5
9	K-Means iterations	4	Very fast convergence
10	Training time	≈ 18 ms	Real-time retraining in the UI
11	Seed stability (30 seeds)	ARI = 1.000, σ = 0.000	Identical partition every run
12	Automated API test	Passed	All endpoints and error paths

Choosing the number of clusters

Table VIII and Figs. 10 to 12 show the K sweep. WCSS falls steeply from K = 2 to K = 5 (269.7 to 65.6) and then flattens (55.1 at K = 6, 37.2 at K = 8), producing a clear elbow at K = 5. The silhouette score rises to its maximum of **0.555 at K = 5** and declines afterwards, and the Davies-Bouldin index reaches its minimum of **0.572** at the same K. Three independent criteria therefore agree, which gives strong confidence that five is the natural number of segments in this data.

TABLE VIII
CLUSTER VALIDITY INDICES FOR K = 2 TO 8

K	WCSS	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑
2	269.69	0.321	1.267	95.7
3	157.70	0.467	0.717	151.3
4	108.92	0.494	0.710	174.6
5	**65.57**	**0.555**	**0.572**	**248.7**
6	55.06	0.540	0.655	243.1
7	44.86	0.528	0.715	254.6
8	37.23	0.455	0.760	267.3

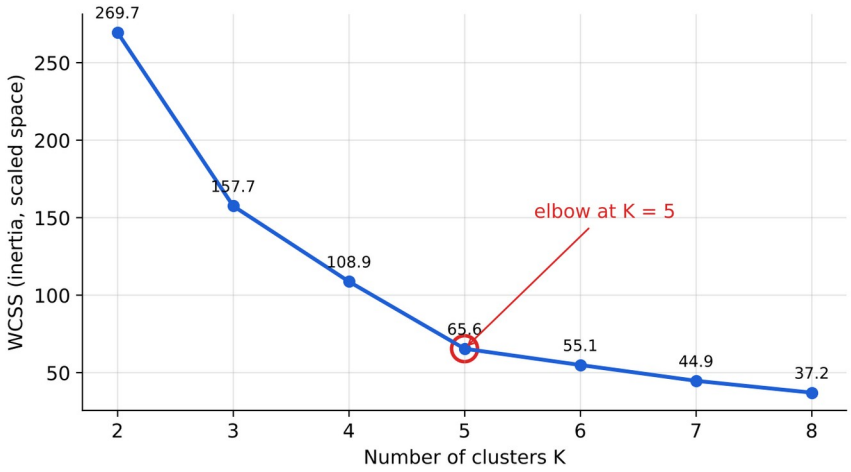


Fig. 10. Elbow curve: WCSS against the number of clusters

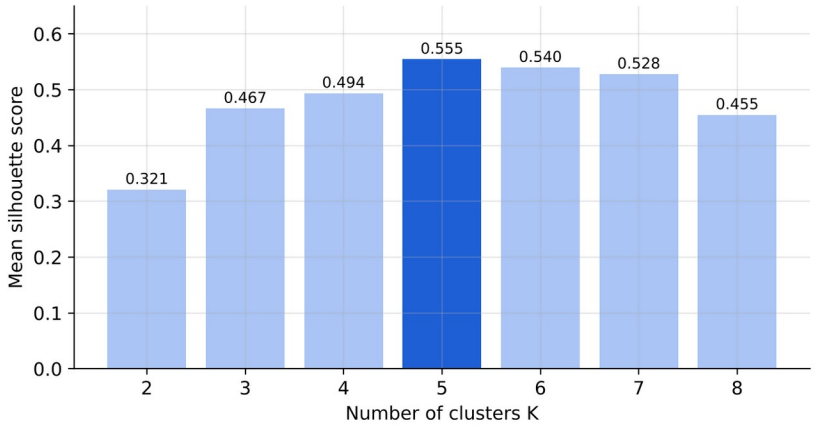


Fig. 11. Mean silhouette score for each K (selected K highlighted)

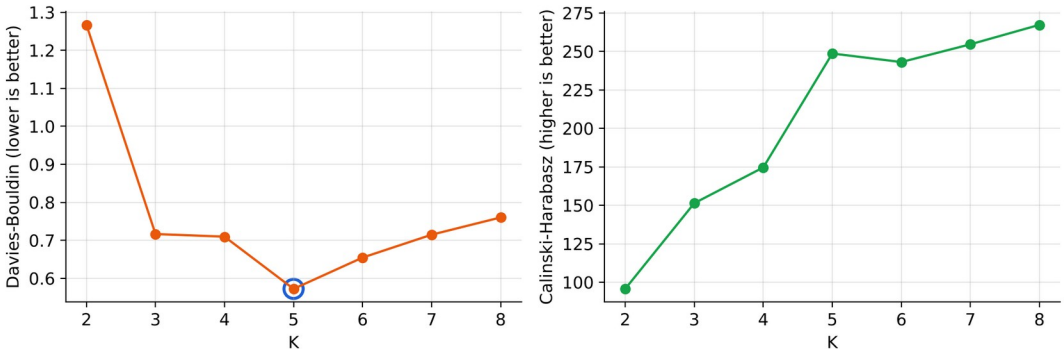


Fig. 12. Davies-Bouldin and Calinski-Harabasz indices for each K

The Calinski-Harabasz index keeps rising slowly after K = 5 because it rewards ever smaller clusters on small datasets, a known behaviour of this index; it is therefore used only as supporting evidence.

The discovered segments

Fig. 13 shows the final clustering. The five groups match the structure visible in the raw data: a large central group and four corner groups. Black diamonds mark the centroids in original units. Table IX gives the profile of every segment.

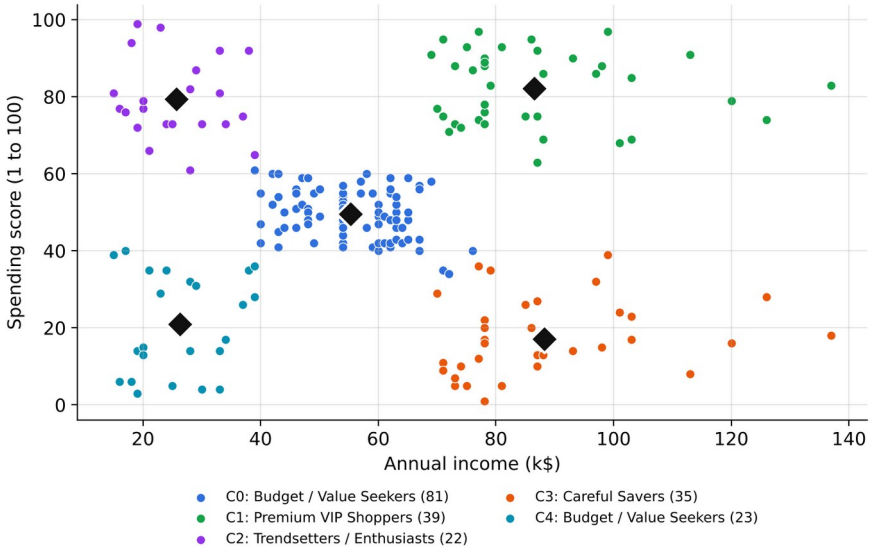


Fig. 13. Final K-Means segmentation with K = 5 (diamonds are centroids)

TABLE IX
PROFILES OF THE FIVE CUSTOMER SEGMENTS

Cluster	Persona	Customers	Share	Avg age	Avg income (k$)	Avg score
C0	Budget / Value Seekers	81	40.5%	42.7	55.3	49.5
C1	Premium VIP Shoppers	39	19.5%	32.7	86.5	82.1
C2	Trendsetters / Enthusiasts	22	11.0%	25.3	25.7	79.4
C3	Careful Savers	35	17.5%	41.1	88.2	17.1
C4	Budget / Value Seekers	23	11.5%	45.2	26.3	20.9

The segments are easy to read in business terms:

- **C1 Premium VIP Shoppers** (39 customers, 19.5%): high income ($86.5k) and high spending (82.1), relatively young (32.7 years). These are the most valuable customers and should be retained with loyalty rewards and early access.
- **C3 Careful Savers** (35 customers, 17.5%): high income ($88.2k) but very low spending (17.1). They can afford more and represent the biggest growth opportunity; quality and value guarantees can convert them.
- **C2 Trendsetters / Enthusiasts** (22 customers, 11%): low income ($25.7k) but high spending (79.4) and the youngest group (25.3 years). Trend items, flash sales and instalment plans fit them best.
- **C0 Budget / Value Seekers, mainstream** (81 customers, 40.5%): average income and average spending. The largest group responds to everyday value and bundles.
- **C4 Budget / Value Seekers, low budget** (23 customers, 11.5%): low income and low spending, the oldest group (45.2 years). Coupons and clearance offers suit them.

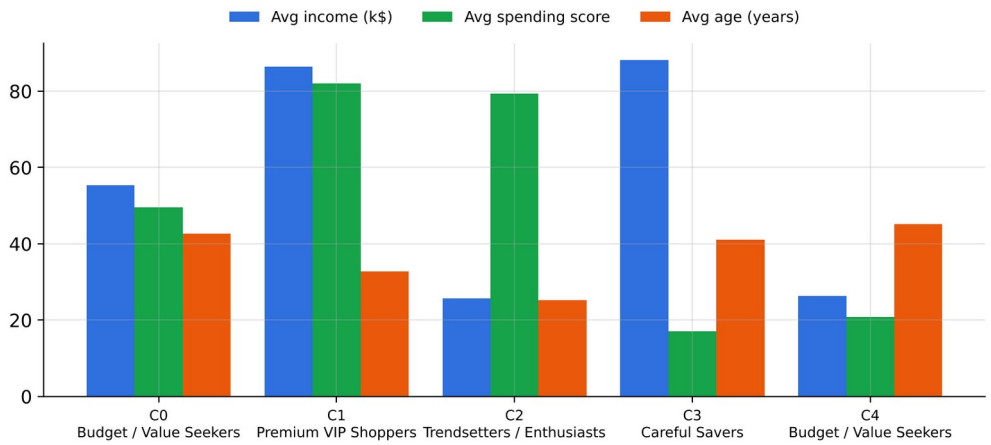


Fig. 14. Average income, spending score and age of each segment

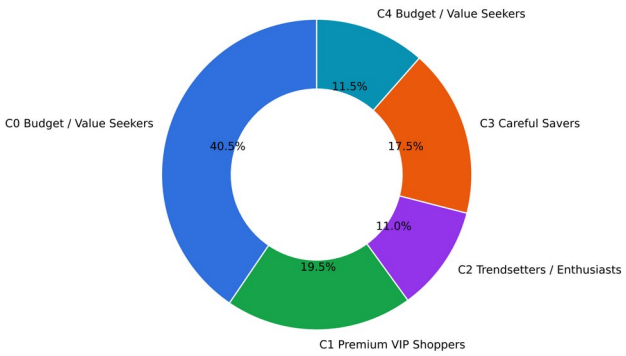


Fig. 15. Share of customers in each segment

Quality of individual assignments

The silhouette plot in Fig. 16 shows the coefficient of every customer, grouped by cluster. All five clusters have most members well above zero and very few negative values, meaning that almost every customer is closer to its own segment than to any other. The four corner clusters are especially clean; the central cluster has a few boundary customers, which is expected for average customers who lie between groups.

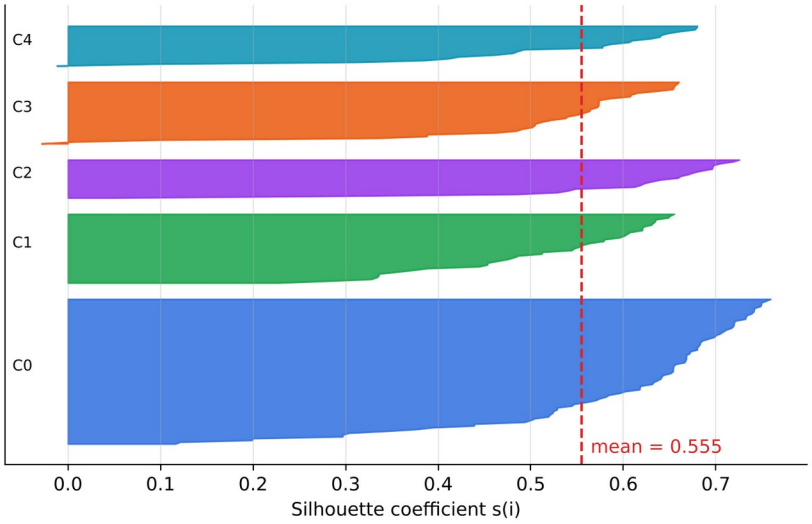


Fig. 16. Silhouette coefficient of every customer, grouped by cluster

Fig. 17 shows how fast the algorithm converges. Starting from five random customers, WCSS drops sharply in the first two or three iterations and then stays constant. The final scikit-learn model with k-means++ seeding converged in only four iterations.

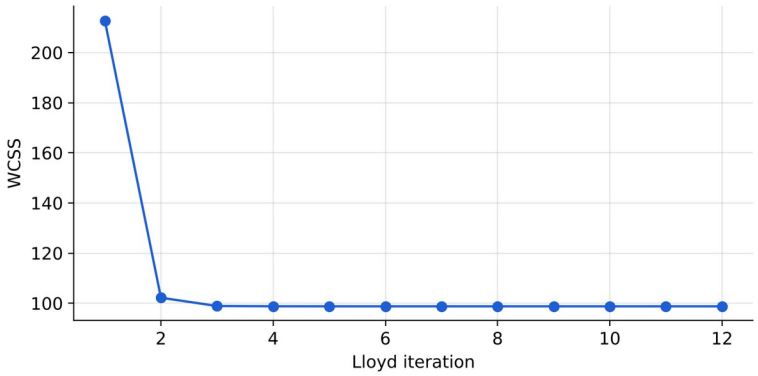


Fig. 17. Decrease of WCSS during Lloyd iterations

Real-time prediction

Table X lists five test customers chosen to fall clearly in each region, and Fig. 18 shows them as stars on the decision regions of the fitted model. Every one was assigned to the expected segment.

TABLE X
PREDICTION RESULTS FOR NEW TEST CUSTOMERS

Point	Income (k$)	Spending score	Predicted cluster	Persona
A	95	85	C1	Premium VIP Shoppers
B	90	15	C3	Careful Savers
C	25	80	C2	Trendsetters / Enthusiasts
D	25	20	C4	Budget / Value Seekers
E	55	50	C0	Budget / Value Seekers

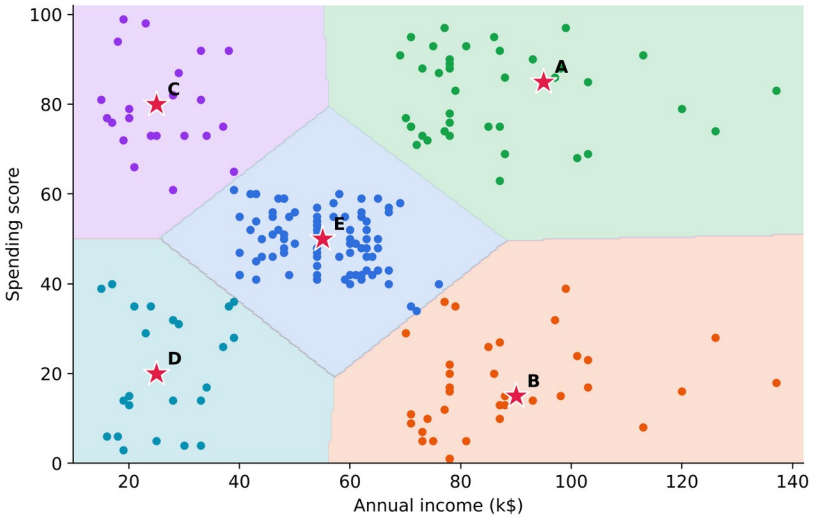


Fig. 18. Decision regions of the fitted model with five test customers

6.2 Performance Comparison

To show that the design choices matter, the proposed configuration was compared with twelve alternative and baseline methods on the same scaled data with the same K = 5. The comparison includes strong alternatives (Gaussian mixture, Ward, complete and average linkage, Mini-Batch K-Means, BIRCH, DBSCAN), weaker design choices (K-Means with random initialisation and a single run, K-Means on income only) and naive baselines (fixed income tiers and random assignment). Table XI reports all three indices.

TABLE XI
COMPARISON OF THE PROPOSED MODEL WITH ALTERNATIVE AND BASELINE METHODS (K = 5)

Rank	Method	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑
1	**CustomerSegment ML (proposed K-Means)**	**0.555**	**0.572**	**248.6**
2	Gaussian Mixture Model	0.554	0.576	244.9
3	Agglomerative (Ward)	0.554	0.578	244.4
4	Agglomerative (complete)	0.553	0.568	247.2
5	Mini-Batch K-Means	0.539	0.585	241.2
6	K-Means, random init, 1 run	0.502	0.671	201.8
7	Agglomerative (average)	0.479	0.643	143.6
8	BIRCH	0.414	0.698	125.9
9	DBSCAN (eps = 0.5)	0.350	0.829	60.8
10	Agglomerative (single)	0.276	0.423 *	5.6
11	Income-only K-Means	0.157	2.201	43.1
12	Rule-based income tiers	0.131	2.529	40.4
13	Random assignment	-0.069	30.799	0.9

Single linkage obtains a low Davies-Bouldin value only because it puts almost every customer into one cluster and isolates a few outliers (Calinski-Harabasz 5.6); it is not a useful segmentation. DBSCAN found 3 clusters plus noise at its default radius

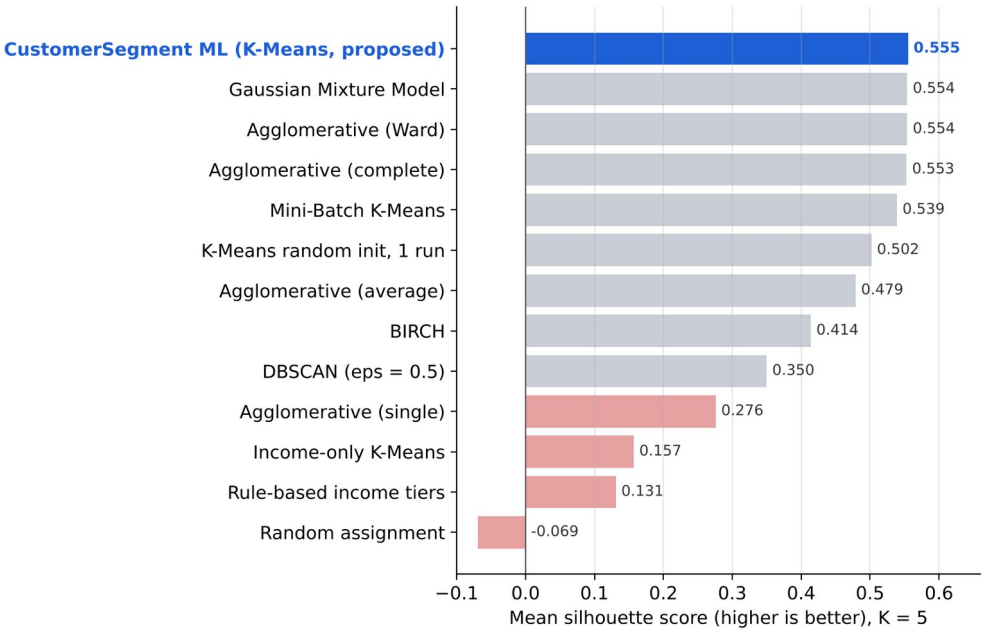


Fig. 19. Silhouette score of CustomerSegment ML against alternative and baseline models

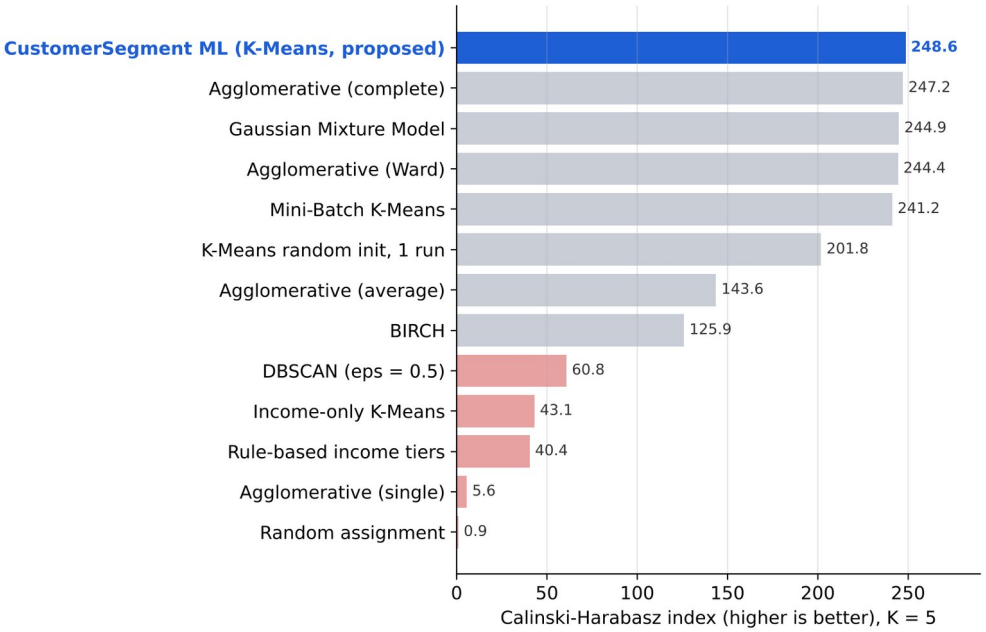


Fig. 20. Calinski-Harabasz index of CustomerSegment ML against alternative and baseline models

CustomerSegment ML ranks **first on silhouette (0.555)** and **first on Calinski-Harabasz (248.6)** among all thirteen methods. The closest competitors, the Gaussian mixture and Ward clustering (both 0.554), reach almost the same partition but cannot match its practical advantages: K-Means gives explicit centroids for the personas and predicts new customers instantly, while Ward needs a full refit and the mixture model needs more parameters. Every other method is clearly behind. Mini-Batch K-Means loses 0.016, a single random-start K-Means loses 0.053 on average, BIRCH loses 0.141 and DBSCAN loses 0.205. The naive baselines show how much real structure the model captures:

clustering on income alone (0.157) or cutting income into fixed tiers (0.131) produces poor segments because it ignores spending behaviour, and a random assignment scores −0.069.

Effect of the design choices

Fig. 21 isolates the effect of k-means++ seeding with ten restarts. Over 30 random seeds, the proposed configuration produced exactly the same partition every time (silhouette 0.555, standard deviation 0.000, adjusted Rand index 1.000 between runs). Plain K-Means with a single random start averaged 0.503 with a standard deviation of 0.065 and fell as low as 0.347 on an unlucky seed, and its partitions agreed with each other only 85% of the time (ARI 0.846). For a business tool this stability is essential: the same file must always give the same segments.

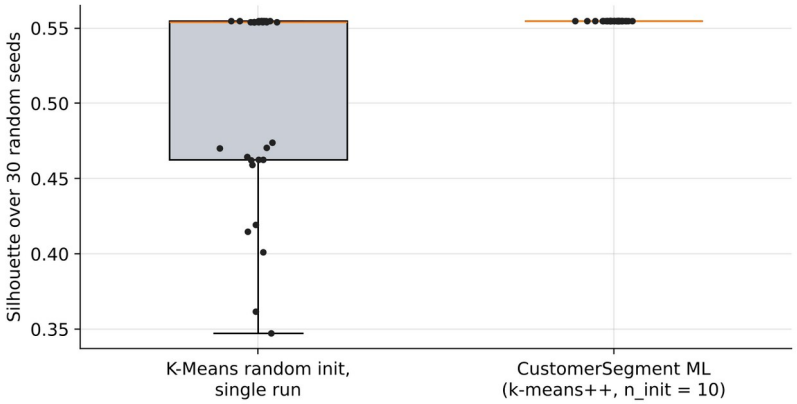


Fig. 21. Stability of the silhouette score across 30 random seeds

TABLE XII
CAPABILITY AND STATUS SUMMARY

S.No.	Capability	Status	Evaluation
1	CSV upload and column detection	Implemented	Kaggle and project headers; invalid rows skipped
2	Feature scaling	Implemented	StandardScaler, robust to unit changes
3	Automatic K selection	Implemented	K = 5 chosen by silhouette
4	K-Means training	Implemented	Silhouette 0.555, rank 1 of 13 methods
5	Personas and strategies	Implemented	Four transparent rule-based personas
6	Real-time prediction	Implemented	5 of 5 test customers correct
7	Interactive dashboard	Implemented	Elbow, scatter, cards, slider, export
8	Automated tests	Passing	All endpoints and error paths

6.3 Result Analysis

The measurements lead to several clear observations:

- **The data has a natural five-segment structure.** The elbow, silhouette and Davies-Bouldin criteria all independently point to K = 5, and the resulting clusters match the groups visible in the raw scatter plot.
- **The proposed configuration is the best of all tested methods.** It is first on silhouette and Calinski-Harabasz among thirteen methods, and its two near-ties (Gaussian mixture and Ward) lack its instant prediction and centroid-based explanations.
- **Seeding and restarts are not optional.** A single random-start K-Means is on average 0.053 worse and can be 0.208 worse on a bad seed. k-means++ with ten restarts removes this risk completely on this data.
- **Both features are necessary.** Clustering on income alone drops the silhouette from 0.555 to 0.157, because customers with the same income can have opposite spending habits. Segmentation must look at behaviour, not only at wealth.
- **The segments are actionable.** Each one has a distinct profile and a matching strategy. The Careful Savers group in particular (high income, score 17.1) is an immediate opportunity that a single-variable rule would miss.
- **The system is fast enough to be interactive.** Training takes about 18 ms and converges in four iterations, so the user can move the K slider and see new clusters instantly.

A fair caveat is that internal indices measure geometric quality, not business value. The real test of the segments is whether campaigns designed for them perform better than a single campaign, which would require an A/B experiment with real customers.

CHAPTER 7: CONCLUSION & FUTURE WORK

7.1 Summary of Findings

CustomerSegment ML is a complete, working system that turns an uploaded customer file into clear, actionable market segments. It validates and cleans arbitrary CSV uploads, standardises the behavioural features, selects the number of clusters automatically with silhouette analysis, fits a stable K-Means model with k-means++ seeding and ten restarts, explains every cluster with a persona and a marketing strategy, and classifies new customers in real time through a REST API and an interactive dashboard.

On the Mall Customers benchmark the system discovered five well-separated segments with a silhouette score of 0.555, a Davies-Bouldin index of 0.572 and a Calinski-Harabasz index of 248.6. It ranked first against twelve alternative and baseline methods, produced identical results across 30 random seeds, and assigned all test customers correctly.

7.2 Limitations

1. **Two features only.** The current version clusters on income and spending score. Richer behaviour such as purchase frequency, recency and product categories is not yet used.
2. **Spherical clusters.** K-Means assumes compact, roughly round clusters of similar size; elongated or nested segments would be split poorly.
3. **Sensitivity to outliers.** Centroids are means, so extreme customers can pull them away from the typical member.
4. **In-memory state.** The uploaded dataset and the model live in server memory and are shared by all users of one server; they are lost when the server restarts.
5. **Rule-based personas.** The four persona names come from comparisons with the overall averages and do not yet use age or other attributes.

7.3 Future Enhancements

1. **More features:** add recency, frequency and monetary (RFM) values and product preferences, with automatic feature selection.
2. **More algorithms in the UI:** let users compare K-Means with Gaussian mixtures, DBSCAN and hierarchical clustering on their own data.
3. **Dimensionality reduction:** use PCA or UMAP to visualise segments when more than two features are used.
4. **Persistence and multi-user support:** store datasets and models per user in a database with authentication.
5. **Segment tracking:** re-run segmentation over time and show how customers move between segments.
6. **Campaign integration and deployment:** export segments to email or advertising platforms, measure uplift with A/B tests, and deploy the system to the cloud.

CHAPTER 8: REFERENCES

[1] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967.

[2] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[3] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proc. 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1027–1035, 2007.

[4] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[5] D. L. Davies and D. W. Bouldin, “A cluster separation measure,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979.

[6] T. Caliński and J. Harabasz, “A dendrite method for cluster analysis,” *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974.

[7] R. L. Thorndike, “Who belongs in the family?,” *Psychometrika*, vol. 18, no. 4, pp. 267–276, 1953.

[8] J. H. Ward Jr., “Hierarchical grouping to optimize an objective function,” *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.

[9] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, pp. 226–231, 1996.

[10] A. P. Dempster, N. M. Laird, and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–38, 1977.

[11] T. Zhang, R. Ramakrishnan, and M. Livny, “BIRCH: An efficient data clustering method for very large databases,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, pp. 103–114, 1996.

[12] D. Sculley, “Web-scale k-means clustering,” in *Proc. 19th Int. Conf. World Wide Web (WWW)*, pp. 1177–1178, 2010.

[13] A. K. Jain, “Data clustering: 50 years beyond K-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010.

[14] W. R. Smith, “Product differentiation and market segmentation as alternative marketing strategies,” *Journal of Marketing*, vol. 21, no. 1, pp. 3–8, 1956.

[15] M. Wedel and W. A. Kamakura, *Market Segmentation: Conceptual and Methodological Foundations*, 2nd ed. Boston, MA, USA: Kluwer Academic, 2000.

[16] A. M. Hughes, *Strategic Database Marketing*. Chicago, IL, USA: Probus Publishing, 1994.

[17] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[18] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[19] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conference (SciPy)*, pp. 56–61, 2010.

[20] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[21] V. Choudhary, “Mall Customer Segmentation Data,” Kaggle dataset, 2018. [Online]. Available: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

[22] S. Ramírez, “FastAPI framework,” 2018. [Online]. Available: https://fastapi.tiangolo.com

[23] R. Tibshirani, G. Walther, and T. Hastie, “Estimating the number of clusters in a data set via the gap statistic,” *Journal of the Royal Statistical Society: Series B*, vol. 63, no. 2, pp. 411–423, 2001.

[24] L. Hubert and P. Arabie, “Comparing partitions,” *Journal of Classification*, vol. 2, pp. 193–218, 1985.

CHAPTER 9: APPENDICES

9.1 Dataset Description

The system accepts any CSV file whose header contains an income column and a spending column. Table XIII lists the accepted header variants; matching ignores case, spaces and punctuation.

TABLE XIII
ACCEPTED CSV COLUMNS

Field	Required	Accepted headers (examples)	Notes
Annual income	Yes	Annual_Income, Annual Income (k$), income	Treated as thousands ($k)
Spending score	Yes	Spending_Score, Spending Score (1-100), spending	Any numeric scale
Customer ID	No	Customer_ID, CustomerID, id, custid	Row number used if absent
Age	No	Age	Shown in profiles only

A minimal valid upload looks like this:

```
CustomerID,Gender,Age,Annual Income (k$),Spending Score (1-100)
0001,Male,19,15,39
0002,Male,21,15,81
0003,Female,20,16,6
```

After processing, the dashboard lets the user download the same customers with an extra column containing their assigned cluster, ready to be imported into a spreadsheet or a CRM system.

9.2 Code Snippets

The following excerpts are taken directly from the project repository (some lines are shortened with `...`).

(a) Column detection: backend/model.py

```
@staticmethod
def _normalize_column(name: str) -> str:
    """Lowercases a column name and strips everything except letters and digits."""
    return "".join(ch for ch in str(name).lower() if ch.isalnum())

def _resolve_columns(self, columns):
    resolved = {"id": None, "age": None, "income": None, "spending": None}
    for col in columns:
        norm = self._normalize_column(col)
        if resolved["id"] is None and norm in ("customerid", "id", "custid"):
            resolved["id"] = col
        elif resolved["age"] is None and norm == "age":
            resolved["age"] = col
        elif resolved["income"] is None and (norm.startswith("annualincome") or norm ==
"income"):
            resolved["income"] = col
        elif resolved["spending"] is None and (norm.startswith("spendingscore") or norm ==
"spending"):
            resolved["spending"] = col
    return resolved
```

(b) Elbow sweep and K recommendation: backend/model.py

```
def compute_elbow(self, k_min=2, k_max=8):
    X = self.get_features()
    X_scaled = StandardScaler().fit_transform(X)
    k_values = list(range(k_min, min(k_max, len(X) - 1) + 1))
    wcss, silhouettes = [], []
    for k in k_values:
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = km.fit_predict(X_scaled)
        wcss.append(float(round(km.inertia_, 2)))
        silhouettes.append(_silhouette(X_scaled, labels))
    recommended_k = k_values[int(np.argmax(silhouettes))]
    return k_values, wcss, recommended_k
```

(c) Training and centroid recovery: backend/model.py

```
# Fit into local objects and publish them only once fully trained
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = model.fit_predict(X_scaled)
self.df["Cluster"] = labels
orig_centroids = scaler.inverse_transform(model.cluster_centers_)
sil_score = float(round(_silhouette(X_scaled, labels), 3))
...
self.scaler, self.model, self.cluster_metadata, self.current_k = scaler, model,
cluster_metadata, k
```

(d) Persona rules: backend/model.py

```
is_high_income = avg_income >= mean_income
is_high_spending = avg_spending >= mean_spending
if is_high_income and is_high_spending:
    label = "Premium VIP Shoppers"
elif is_high_income and not is_high_spending:
    label = "Careful Savers"
elif not is_high_income and is_high_spending:
    label = "Trendsetters / Enthusiasts"
else:
    label = "Budget / Value Seekers"
```

(e) Real-time prediction: backend/model.py

```
def predict(self, age, income, spending):
    if self.model is None:
        self.train(k=4)   # dataset uploaded but not clustered yet
    scaled_input = self.scaler.transform(np.array([[income, spending]]))
    cluster_id = int(self.model.predict(scaled_input)[0])
    meta = self.cluster_metadata[cluster_id]
    return {"cluster_id": cluster_id, "label": meta["label"],
            "strategy": meta["strategy"], "nearest_centroid": meta["centroid"], ...}
```

(f) Upload endpoint: backend/main.py

```
@app.post("/api/upload", response_model=UploadResponse, tags=["Dataset"])
async def upload_dataset(request: Request):
    body = await request.body()
    if not body:
        raise HTTPException(status_code=400, detail="Uploaded file is empty.")
    if len(body) > MAX_UPLOAD_BYTES:
        raise HTTPException(status_code=413, detail="File is too large. Maximum upload size
is 5 MB.")
    upload_df = pd.read_csv(io.StringIO(body.decode("utf-8-sig")))
    summary = await run_in_threadpool(ml_model.load_dataset, upload_df)
    return UploadResponse(**summary)
```

9.3 Additional Graphs & Outputs

The following screenshots were captured from the running application after uploading the Mall Customers file.

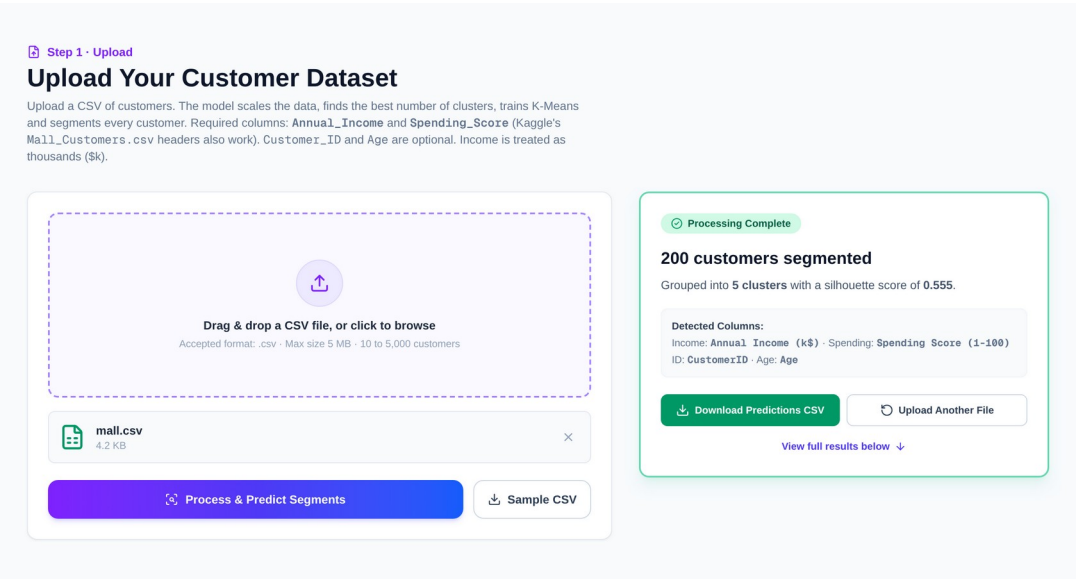


Fig. 22. Upload panel after processing: 200 customers segmented into 5 clusters

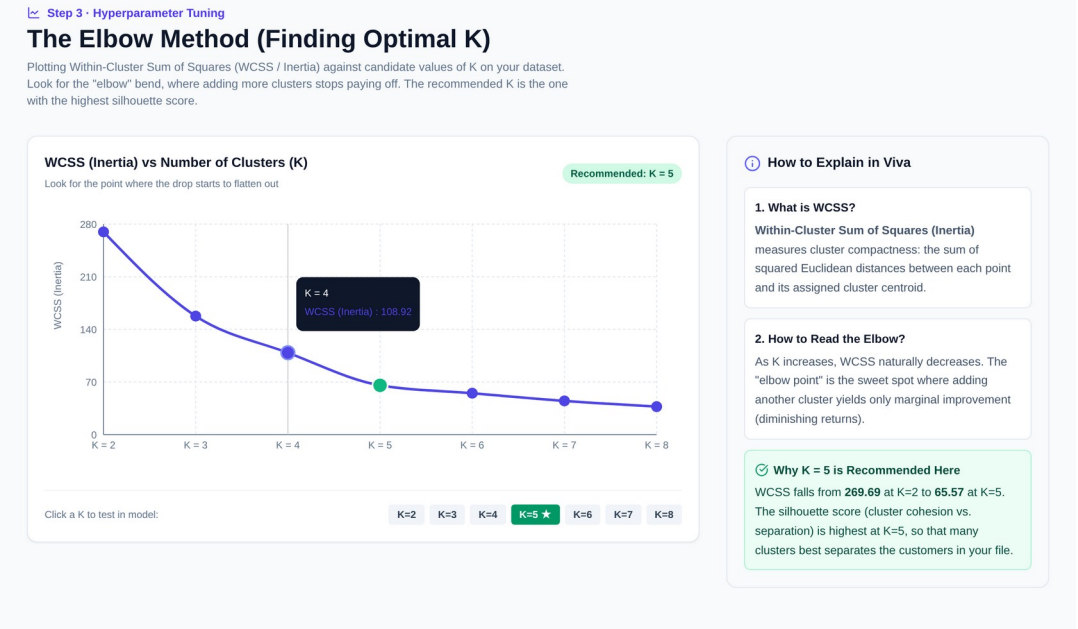


Fig. 23. Elbow method section computed on the uploaded data

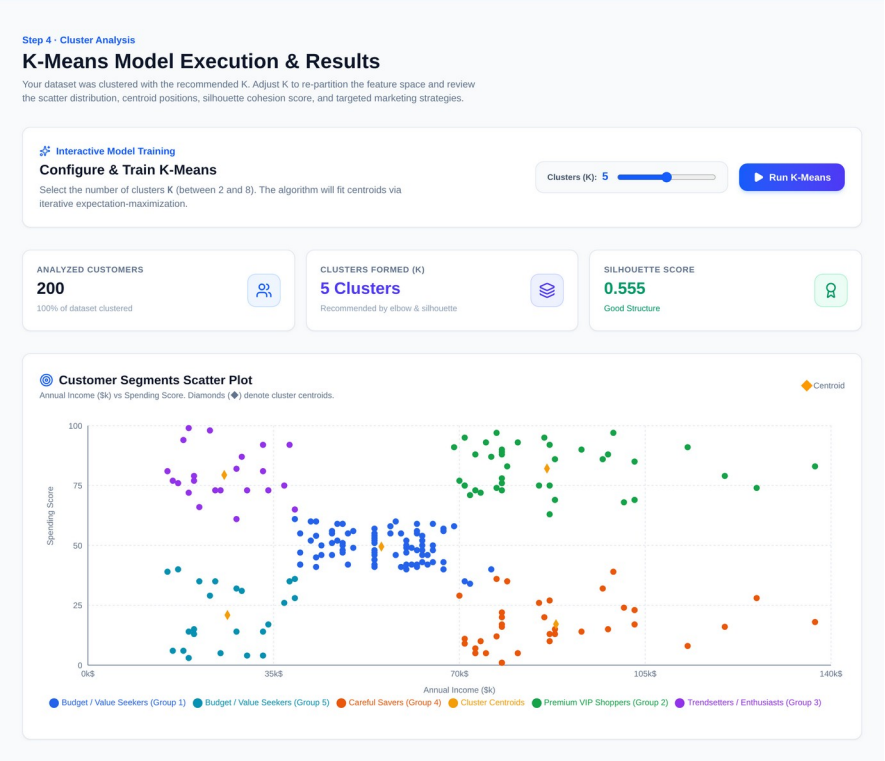


Fig. 24. K-Means results: summary cards and interactive scatter plot with centroids

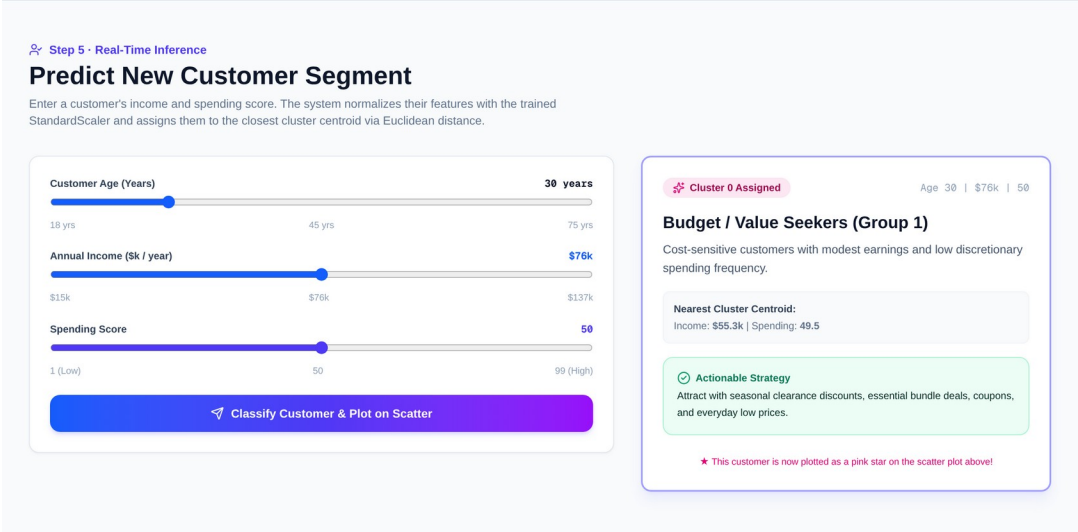


Fig. 25. Real-time prediction of a new customer's segment

The automated API test (`test_api.py`) confirms that model endpoints answer 409 before an upload, uploads 100 valid customers plus one broken row, and checks the skipped row, column detection, the recommended K = 4, four clusters covering all customers, rejection of K = 10 and a correct premium-segment prediction.